0300. Longest Increasing Subsequence

Medium · 1-D Dynamic Programming
leetcode.com/problems/longest-increasing-subsequence

Problem

- Return the length of the longest strictly increasing subsequence.

Intuition

- O(n^2) DP: dp[i] = 1 + max dp[j] for j < i with nums[j] < nums[i].
- O(n log n) "patience sorting": tails[k] = smallest possible tail of an increasing subsequence of length k+1. For each x, replace the first tail >= x (binary search) or append.

Brute Force

- All subsequences: O(2^n).

Optimal Approach

- Tails = empty.
- For x: pos = lower_bound(tails, x); if pos == size append else tails[pos] = x.
- Answer = tails size.

Worked Example

```
[10,9,2,5,3,7,101,18].

10  -> [10]
9   -> [9]
2   -> [2]
5   -> [2,5]
3   -> [2,3]
7   -> [2,3,7]
101 -> [2,3,7,101]
18  -> [2,3,7,18]   length 4
```

Java Solution

```java
public class LongestIncreasingSubsequence {
    public int lengthOfLIS(int[] nums) {
        int[] tails = new int[nums.length];
        int size = 0;
        for (int x : nums) {
            int lo = 0, hi = size;
            while (lo < hi) {
                int mid = (lo + hi) >>> 1;
                if (tails[mid] < x) lo = mid + 1;
                else hi = mid;
            }
            tails[lo] = x;
            if (lo == size) size++;
        }
        return size;
    }

    public int lengthOfLISQuadratic(int[] nums) {
        int[] dp = new int[nums.length];
        int best = 0;
        for (int i = 0; i < nums.length; i++) {
            dp[i] = 1;
            for (int j = 0; j < i; j++) if (nums[j] < nums[i]) dp[i] = Math.max(dp[i], dp[j] + 1);
            best = Math.max(best, dp[i]);
        }
        return best;
    }
}
```

Complexity

- Time: O(n log n). Space: O(n).

Pitfalls

- Tails is not itself an LIS; only its length is meaningful.
- Use lower bound (>=) for strictly increasing.